

Pontifícia Universidade Católica do Rio Grande do Sul

Luis Felipe Flores Acosta

João Vitor Amaral Rangel

Vinicius

Porto Alegre

2026

DESCRIÇÃO DA ARQUITETURA

O servidor HTTP/1.1 foi implementado em Python 3 diretamente sobre sockets TCP, sem a utilização de frameworks ou bibliotecas que implementem o protocolo HTTP no lado servidor. Dessa forma, tanto a interpretação das requisições quanto a construção das respostas HTTP são realizadas pela própria aplicação. O programa recebe, por argumentos de linha de comando, a porta TCP utilizada para atendimento e o diretório raiz (root) que contém os arquivos disponibilizados aos clientes. O socket principal utiliza IPv4/TCP (AF_INET e SOCK_STREAM) e realiza bind no endereço 0.0.0.0, permitindo que o servidor receba conexões provenientes de outras máquinas da rede, e não apenas da própria máquina por localhost.

Após a inicialização, o socket servidor entra em estado de escuta por meio de listen() e permanece em um laço executando accept(). O socket principal é responsável somente por aceitar novas conexões TCP. Quando uma conexão é estabelecida, o socket específico daquele cliente é entregue a uma rotina separada de atendimento, enquanto o socket principal retorna imediatamente ao accept() para continuar disponível para novas conexões.

Cada conexão mantém um buffer próprio de bytes recebidos. Essa característica é necessária porque o TCP fornece um fluxo contínuo de bytes e não preserva os limites de uma requisição HTTP: uma chamada a recv() pode retornar apenas parte de uma requisição, uma requisição completa ou ainda dados pertencentes a mais de uma requisição. A implementação, portanto, acumula os dados recebidos até encontrar a sequência \r\n\r\n, que delimita o final dos cabeçalhos HTTP. Os bytes que excedem o término da requisição atual permanecem no buffer para serem utilizados no processamento da requisição seguinte.

O processamento da requisição é realizado manualmente. A aplicação interpreta a linha de requisição, separando método, request-target e versão do protocolo, e posteriormente processa os cabeçalhos. O servidor aceita HTTP/1.1, verifica a presença do cabeçalho obrigatório Host, valida a estrutura dos demais cabeçalhos e controla o Content-Length quando presente. Os métodos implementados são GET e HEAD; outros métodos resultam em 405 Method Not Allowed, acompanhado do cabeçalho Allow: GET, HEAD.

Após a interpretação da requisição, o caminho solicitado é decodificado e associado ao diretório raiz informado na inicialização. Antes de acessar o sistema de arquivos, o servidor obtém o caminho real do recurso e verifica se ele permanece contido dentro desse diretório. Essa etapa impede que sequências como ../, inclusive quando representadas por percent-encoding, permitam acesso a arquivos externos ao diretório publicado. Caso o caminho ultrapasse o diretório raiz, a requisição é respondida com 403 Forbidden. A implementação

também diferencia arquivos inexistentes, requisições malformadas e métodos não suportados, produzindo os códigos 404 Not Found, 400 Bad Request e 405 Method Not Allowed, respectivamente.

As respostas também são construídas manualmente. O servidor gera a linha de status HTTP/1.1, seguida dos cabeçalhos Date, Server, Content-Length, Content-Type e Connection, utilizando `\r\n` como separador conforme a estrutura das mensagens HTTP. Para requisições GET, o conteúdo do arquivo é enviado após os cabeçalhos; para HEAD, são enviados os mesmos cabeçalhos correspondentes ao recurso, inclusive seu Content-Length, porém sem o corpo da resposta.

ESTRATÉGIA DE CONCORRÊNCIA

A estratégia adotada para concorrência foi uma thread por conexão TCP (thread-per-connection). A thread principal do servidor permanece responsável por aceitar novos clientes. Para cada resultado de `accept()`, é criada uma nova `threading.Thread`, cuja função é executar todo o atendimento daquela conexão por meio da rotina `handle_client()`. As threads são configuradas como daemon, de forma que não impeçam o encerramento do processo principal.

Essa estratégia foi escolhida porque as operações realizadas pelo servidor são predominantemente de entrada e saída, como aguardar dados em `recv()`, enviar respostas pelo socket e ler arquivos do disco. Caso todo o processamento fosse realizado sequencialmente na thread principal, um cliente que enviasse a requisição lentamente poderia manter o servidor bloqueado enquanto os demais clientes aguardassem atendimento.

Com uma thread independente para cada conexão, o bloqueio de uma chamada `recv()` afeta somente a thread responsável por aquele cliente. Simultaneamente, a thread principal pode continuar aceitando novas conexões e as demais threads podem processar outras requisições normalmente.

Essa característica foi verificada experimentalmente no teste de atendimento simultâneo. Uma máquina manteve uma requisição lenta ainda em andamento, enquanto outra conexão foi aceita e teve sua requisição respondida com 200 OK antes da conclusão da primeira. Dessa forma, uma conexão lenta não impediu o atendimento das demais.

A escolha por uma thread por conexão também é adequada ao escopo do trabalho por apresentar uma implementação relativamente simples e permitir que cada conexão possua seu próprio estado, incluindo socket, endereço do cliente, buffer e controle da conexão persistente. Embora arquiteturas baseadas em I/O assíncrono possam apresentar melhor escalabilidade para grandes quantidades de conexões simultâneas, essa complexidade adicional não é necessária para o

cenário proposto, cujo objetivo é demonstrar de forma clara o funcionamento do HTTP/1.1 sobre TCP e comprovar o atendimento simultâneo de clientes distintos.

CONEXÕES PERSISTENTES

Como o servidor implementa HTTP/1.1, as conexões são persistentes por padrão. Após enviar uma resposta, a thread responsável pelo cliente não encerra imediatamente o socket. Ela retorna ao processamento do buffer e aguarda uma nova requisição sobre a mesma conexão TCP. Dessa maneira, diversas requisições podem utilizar um único handshake TCP.

O buffer é preservado entre as iterações, permitindo que dados pertencentes à próxima requisição, caso já tenham sido recebidos pelo socket, sejam mantidos e posteriormente processados sem perda de informação.

O encerramento da conexão ocorre quando o cliente envia `Connection: close`, quando o cliente fecha o socket ou quando ocorre inatividade por 5 segundos. Esse timeout evita que uma conexão abandonada permaneça indefinidamente ocupando uma thread e recursos do servidor. Portanto, cada thread permanece ativa durante a vida da conexão TCP e pode processar várias requisições HTTP sucessivas antes de ser encerrada.

Essa organização separa claramente as responsabilidades da aplicação: o socket principal aceita novas conexões; uma thread independente gerencia cada conexão; o parser interpreta o fluxo de bytes recebido; a lógica de resolução valida e localiza o recurso solicitado; e, por fim, o servidor constrói e envia manualmente a resposta HTTP/1.1.

A arquitetura adotada atende, portanto, aos requisitos do trabalho de comunicação diretamente por sockets TCP, atendimento simultâneo de múltiplos clientes e suporte a conexões persistentes no HTTP/1.1.

Tabela de conformidade

Para verificar a conformidade do servidor, foram realizadas requisições que exercitam todos os códigos de status obrigatórios. Os testes foram executados com o servidor disponível em 127.0.0.1:8080. A Tabela 1 apresenta os comandos utilizados e as respostas obtidas.

Tabela 1 – Conformidade dos códigos de status HTTP obrigatórios

Código esperado	Requisição utilizada	Resposta obtida
200 OK	<code>curl.exe -i http://127.0.0.1:8080/index.html</code>	200 OK. O servidor retornou corretamente o arquivo solicitado.
400 Bad Request	<code>curl.exe -i --http1.1 -H "Host:" http://127.0.0.1:8080/</code>	400 Bad Request. A requisição foi rejeitada por não apresentar o cabeçalho Host, obrigatório no HTTP/1.1.
403 Forbidden	<code>curl.exe -i --path-as-is http://127.0.0.1:8080/../../server.py</code>	403 Forbidden. O servidor impediu o acesso a um arquivo localizado fora do diretório raiz configurado.
404 Not Found	<code>curl.exe -i http://127.0.0.1:8080/arquivo-inexistente.txt</code>	404 Not Found. O servidor informou corretamente que o recurso solicitado não existe.
405 Method Not Allowed	<code>curl.exe -i -X POST http://127.0.0.1:8080/index.html</code>	405 Method Not Allowed. A resposta também apresentou o cabeçalho Allow: GET, HEAD.

Fonte: Elaborado pelos autores (2026).

DEMONSTRAÇÃO DE SEGURANÇA

Para verificar a proteção implementada contra travessia de diretório (directory traversal), foram realizadas três tentativas de acessar o arquivo `server.py`, localizado fora do diretório raiz `www` configurado para o servidor.

Os testes utilizaram diferentes representações do caminho de travessia, incluindo a sequência convencional `"../"` e versões utilizando percent-encoding. Em todos os casos foi utilizada a opção `--path-as-is` do `curl`, impedindo que o próprio cliente normalize o caminho antes de enviá-lo ao servidor.

A Figura 1 apresenta o log do servidor durante a execução dos três testes. É possível observar que as três conexões foram aceitas e que todas as tentativas de acesso ao arquivo server.py foram rejeitadas com o código HTTP 403 Forbidden.

```
PS C:\Users\jvtot\OneDrive\Área de Trabalho\05.10.26\T1---Laborat-rio-de-Redes-de-Computadores> python server.py
--port 8080 --root ./www
Servidor HTTP/1.1 iniciado
Endereço: 0.0.0.0:8080
Root: C:\Users\jvtot\OneDrive\Área de Trabalho\05.10.26\T1---Laborat-rio-de-Redes-de-Computadores\www
[127.0.0.1:64024] conexão aberta
[127.0.0.1:64024] GET ../../server.py -> 403 Forbidden
[127.0.0.1:64024] conexão encerrada
[127.0.0.1:64028] conexão aberta
[127.0.0.1:64028] GET /%2E%2E/%2E%2E/server.py -> 403 Forbidden
[127.0.0.1:64028] conexão encerrada
[127.0.0.1:64031] conexão aberta
[127.0.0.1:64031] GET /..%2F..%2Fserver.py -> 403 Forbidden
[127.0.0.1:64031] conexão encerrada
```

Figura 1 – Log do servidor durante as três tentativas de travessia de diretório.

Fonte: Elaborado pelos autores (2026).

O primeiro teste utilizou diretamente segmentos "../" no caminho solicitado:

curl.exe -v --path-as-is <http://127.0.0.1:8080/../../server.py>

A Figura 2 apresenta a requisição enviada pelo curl e a resposta produzida pelo servidor.

```
PS C:\Users\jvtot\OneDrive\Área de Trabalho\05.10.26\T1---Laborat-rio-de-Redes-de-Computadores> curl.exe -v --path-as-is http://127.0.0.1:8080/../../server.py
* Trying 127.0.0.1:8080...
* Established connection to 127.0.0.1 (127.0.0.1 port 8080) from 127.0.0.1 port 64024
* using HTTP/1.x
> GET ../../server.py HTTP/1.1
> Host: 127.0.0.1:8080
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 403 Forbidden
< Date: Tue, 06 Oct 2026 00:06:21 GMT
< Server: Grupo-7
< Content-Length: 115
< Content-Type: text/html; charset=utf-8
< Connection: keep-alive
<
<!doctype html>
<html lang="pt-BR"><meta charset="utf-8"><title>403 Forbidden</title><h1>403 Forbidden</h1></html>
* Connection #0 to host 127.0.0.1:8080 left intact
```

Figura 2 – Tentativa de travessia utilizando segmentos "../".

Fonte: Elaborado pelos autores (2026).

Como pode ser observado na Figura 2, o caminho ../../server.py foi enviado sem normalização e chegou efetivamente ao servidor. A aplicação identificou que a resolução desse caminho resultaria em acesso a um arquivo localizado fora do diretório raiz permitido e respondeu:

HTTP/1.1 403 Forbidden

O segundo teste utilizou percent-encoding para representar os caracteres de ponto. A sequência "%2E" corresponde ao caractere ".", de modo que "%2E%2E" representa "..". A requisição utilizada foi:

```
curl.exe -v --path-as-is http://127.0.0.1:8080/%2E%2E/%2E%2E/server.py
```

A Figura 3 apresenta o resultado desse teste.



```
PS C:\Users\jvtot\OneDrive\Área de Trabalho\05.10.26\T1---Laborat-rio-de-Redes-de-Computadores> curl.exe -v --path-as-is http://127.0.0.1:8080/%2E%2E/%2E%2E/server.py
* Trying 127.0.0.1:8080...
* Established connection to 127.0.0.1 (127.0.0.1 port 8080) from 127.0.0.1 port 64028
* using HTTP/1.x
> GET /%2E%2E/%2E%2E/server.py HTTP/1.1
> Host: 127.0.0.1:8080
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 403 Forbidden
< Date: Tue, 06 Oct 2025 00:06:26 GMT
< Server: Grupo-7
< Content-Length: 115
< Content-Type: text/html; charset=utf-8
< Connection: keep-alive
<
<!doctype html>
<html lang="pt-BR"><meta charset="utf-8"><title>403 Forbidden</title><h1>403 Forbidden</h1></html>
* Connection #0 to host 127.0.0.1:8080 left intact
```

Figura 3 – Tentativa de travessia utilizando pontos codificados por percent-encoding.

Fonte: Elaborado pelos autores (2026).

Conforme apresentado na Figura 3, mesmo com os segmentos de travessia codificados como %2E%2E, o servidor realizou a decodificação do caminho antes de validá-lo. Após identificar que o recurso solicitado estaria fora do diretório www, a requisição foi novamente rejeitada com:

HTTP/1.1 403 Forbidden

No terceiro teste, foram mantidos os segmentos "..", porém as barras utilizadas para avançar entre diretórios foram codificadas como "%2F". A requisição enviada foi:

```
curl.exe -v --path-as-is http://127.0.0.1:8080/..%2F..%2Fserver.py
```

A Figura 4 apresenta a resposta obtida.

```

PS C:\Users\jvtot\OneDrive\Área de Trabalho\05.10.26\T1---Laborat-rio-de-Redes-de-Computadores> curl.exe -v --path-as-is http://127.0.0.1:8080/..%2f..%2fserver.py
* Trying 127.0.0.1:8080...
* Established connection to 127.0.0.1 (127.0.0.1 port 8080) from 127.0.0.1 port 64031
* using HTTP/1.x
> GET /..%2f..%2fserver.py HTTP/1.1
> Host: 127.0.0.1:8080
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 403 Forbidden
< Date: Tue, 06 Oct 2026 00:06:34 GMT
< Server: Grupo-7
< Content-Length: 115
< Content-Type: text/html; charset=utf-8
< Connection: keep-alive
<
<!doctype html>
<html lang="pt-BR"><meta charset="utf-8"><title>403 Forbidden</title><h1>403 Forbidden</h1></html>
* Connection #0 to host 127.0.0.1:8080 left intact

```

Figura 4 – Tentativa de travessia utilizando barras codificadas por percent-encoding.

Fonte: Elaborado pelos autores (2026).

Novamente, o servidor decodificou o request-target antes de realizar a validação de segurança. O caminho resultante corresponderia a uma tentativa de sair do diretório raiz configurado, fazendo com que o acesso fosse bloqueado e a aplicação retornasse:

HTTP/1.1 403 Forbidden

Os três testes demonstram que a proteção contra travessia de diretório não depende apenas da identificação textual da sequência "../". O servidor decodifica o caminho solicitado antes de verificar sua localização real no sistema de arquivos e confirma que o recurso permanece contido dentro do diretório raiz configurado.

Dessa forma, tanto uma tentativa direta de travessia quanto variações utilizando percent-encoding foram rejeitadas. Em nenhuma das três situações o conteúdo do arquivo server.py foi exposto ao cliente, e todas as requisições receberam corretamente a resposta 403 Forbidden.

CAPTURA DE UMA TRANSAÇÃO COMPLETA

Para demonstrar o funcionamento completo de uma transação HTTP/1.1 sobre TCP entre máquinas distintas, foi realizada uma requisição GET a partir da máquina cliente de endereço IP 10.115.242.247 para o servidor localizado em 10.132.242.122, utilizando a porta TCP 8080.

A requisição utilizada no cliente foi:

```
curl.exe -v -H "Connection: close" http://10.132.242.122:8080/index.html
```

O cabeçalho Connection: close foi utilizado para que, após o envio da resposta HTTP, a conexão TCP fosse encerrada de forma explícita, permitindo visualizar na mesma captura o estabelecimento da conexão, a troca de dados HTTP e seu encerramento.

A Figura 5 apresenta, no Wireshark, a sequência completa de pacotes correspondente a essa transação.

No.	Time	Source	Destination	Protocol	Length	Info
1523	37.330961500	10.115.242.247	10.132.242.122	TCP	54	60771 → 8080 [ACK] Seq=310 Ack=287 Win=65024 Len=0
1524	37.330963100	10.115.242.247	10.132.242.122	TCP	54	60771 → 8080 [FIN, ACK] Seq=310 Ack=287 Win=65024 Len=0
1525	37.331056500	10.132.242.122	10.115.242.247	TCP	54	8080 → 60771 [ACK] Seq=287 Ack=311 Win=65024 Len=0
1526	37.335111700	10.132.242.122	10.115.242.247	TCP	54	8080 → 60770 [FIN, ACK] Seq=3679988 Ack=1107 Win=64256 Len=0
1527	37.345780300	10.115.242.247	10.132.242.122	TCP	54	60770 → 8080 [ACK] Seq=1107 Ack=3679987 Win=1048320 Len=0
1528	37.345781000	10.115.242.247	10.132.242.122	TCP	54	60770 → 8080 [FIN, ACK] Seq=1107 Ack=3679987 Win=1048320 Len=0
1529	37.345837300	10.132.242.122	10.115.242.247	TCP	54	8080 → 60770 [ACK] Seq=3679987 Ack=1108 Win=64256 Len=0
1539	38.918975200	10.115.242.247	10.132.242.122	TCP	66	60811 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
1540	38.919162700	10.132.242.122	10.115.242.247	TCP	66	8080 → 60811 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
1541	38.927655900	10.115.242.247	10.132.242.122	TCP	54	60811 → 8080 [ACK] Seq=1 Ack=1 Win=65280 Len=0
1542	38.954638100	10.115.242.247	10.132.242.122	HTTP	166	GET /index.html HTTP/1.1
1543	38.956210800	10.132.242.122	10.115.242.247	HTTP	826	HTTP/1.1 200 OK (text/html)
1544	38.956587500	10.132.242.122	10.115.242.247	TCP	54	8080 → 60811 [FIN, ACK] Seq=773 Ack=113 Win=65280 Len=0
1545	38.966349100	10.115.242.247	10.132.242.122	TCP	54	60811 → 8080 [ACK] Seq=113 Ack=774 Win=64512 Len=0
1546	39.897336000	10.132.242.122	10.132.242.122	TCP	54	60811 → 8080 [FIN, ACK] Seq=113 Ack=774 Win=64512 Len=0
1547	39.837643900	10.132.242.122	10.115.242.247	TCP	54	8080 → 60811 [ACK] Seq=774 Ack=114 Win=65280 Len=0

Figura 5 – Captura Wireshark da transação HTTP/1.1 completa entre o cliente 10.115.242.247 e o servidor 10.132.242.122.

Fonte: Elaborado pelos autores (2026).

A comunicação inicia-se com o estabelecimento da conexão TCP por meio do handshake de três vias. Na captura, essa etapa corresponde aos pacotes 1539, 1540 e 1541:

Pacote 1539 – SYN:

o cliente 10.115.242.247 inicia a conexão TCP em direção ao servidor 10.132.242.122 na porta 8080.

Pacote 1540 – SYN, ACK:

o servidor confirma o recebimento da solicitação de conexão e responde ao cliente.

Pacote 1541 – ACK:

o cliente confirma a resposta do servidor, concluindo o handshake TCP. A partir desse momento, a conexão está estabelecida e pode transportar os dados da aplicação.

Após o estabelecimento da conexão, o pacote 1542 transporta a requisição HTTP enviada pelo cliente:

```
GET /index.html HTTP/1.1
```

Nesse pacote também são enviados os cabeçalhos HTTP da requisição, incluindo o cabeçalho Host correspondente ao servidor acessado.

O pacote 1543 contém a resposta enviada pelo servidor:

```
HTTP/1.1 200 OK
```

O código 200 OK demonstra que o arquivo index.html foi localizado e atendido corretamente. Como o recurso possui tamanho reduzido, a resposta HTTP e o conteúdo solicitado puderam ser transportados na captura sem a necessidade de uma longa sequência de segmentos contendo dados da aplicação.

Como evidência complementar, a Figura 6 apresenta a mesma transação observada pelo lado da máquina cliente. A saída do curl permite confirmar o endereço IP do servidor acessado, a requisição GET enviada, o cabeçalho Connection: close e a resposta HTTP/1.1 200 OK recebida.

```

PS C:\Windows\system32> curl.exe -v -H "Connection: close" http://10.132.242.122:8080/index.html
* Trying 10.132.242.122:8080...
* Established connection to 10.132.242.122 (10.132.242.122 port 8080) from 10.115.242.247 port 60811
* using HTTP/1.x
> GET /index.html HTTP/1.1
> Host: 10.132.242.122:8080
> User-Agent: curl/8.21.0
> Accept: */*
> Connection: close
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Tue, 06 Oct 2026 00:28:35 GMT
< Server: Grupo-7
< Content-Length: 619
< Content-Type: text/html; charset=utf-8
< Connection: close
<
<!doctype html>
<html lang="pt-BR">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>T1 - Redes</title>
  <link rel="stylesheet" href="/style.css">
</head>
<body>
  <main>
    <h1>Servidor do Grupo 7 funcionando</h1>
    <p>Esta página foi entregue por um servidor HTTP/1.1 feito com sockets TCP.</p>
    
    <p id="status">Carregando o JavaScript...</p>
  </main>
  <script src="/script.js"></script>
</body>
</html>
* shutting down connection #0

```

Figura 6 – Requisição GET realizada pela máquina cliente 10.115.242.247 ao servidor 10.132.242.122 e resposta HTTP/1.1 200 OK obtida.

Fonte: Elaborado pelos autores (2026).

Na Figura 6, observa-se que o cliente estabelece conexão com 10.132.242.122 na porta 8080 e envia a requisição GET /index.html HTTP/1.1. O cabeçalho Host identifica o servidor de destino e o cabeçalho Connection: close solicita o encerramento da conexão após a resposta. O servidor retorna HTTP/1.1 200 OK, juntamente com os cabeçalhos da resposta e o conteúdo do arquivo index.html, confirmando o atendimento bem-sucedido da requisição.

Após a entrega da resposta, inicia-se o encerramento ordenado da conexão TCP. Os pacotes 1544 a 1547 correspondem à troca de segmentos utilizada para finalizar a comunicação entre cliente e servidor. Essa etapa ocorre porque a requisição foi enviada com Connection: close, indicando que a conexão não deveria permanecer aberta após a resposta.

Dessa forma, a captura permite identificar todas as etapas de uma transação HTTP/1.1 sobre TCP:

1. estabelecimento da conexão TCP por meio do handshake SYN, SYN-ACK e ACK;
2. envio da requisição GET pelo cliente;
3. envio da resposta HTTP/1.1 200 OK pelo servidor;
4. encerramento ordenado da conexão TCP.

A captura também comprova que a requisição foi realizada por uma máquina distinta daquela em que o servidor estava sendo executado, uma vez que o cliente possui o endereço 10.115.242.247 e o servidor o endereço 10.132.242.122.

EVIDÊNCIA DE ATENDIMENTO SIMULTÂNEO

Para verificar se o servidor é capaz de atender múltiplas conexões simultaneamente, foi realizado um teste no qual uma das conexões foi mantida propositalmente lenta enquanto uma segunda requisição era enviada ao servidor.

Inicialmente, a máquina 10.115.242.247 estabeleceu uma conexão com o servidor e iniciou o envio gradual dos cabeçalhos HTTP, mantendo essa conexão aberta durante o teste. Antes que essa primeira requisição fosse concluída, uma segunda conexão foi estabelecida a partir da máquina 10.132.242.122, que solicitou o recurso index.html.

A Figura 7 apresenta o tráfego capturado pelo Wireshark durante o experimento. A captura permite observar comunicações ocorrendo no mesmo intervalo de tempo, evidenciando que o servidor continuou recebendo e processando tráfego enquanto a primeira conexão permanecia ativa.

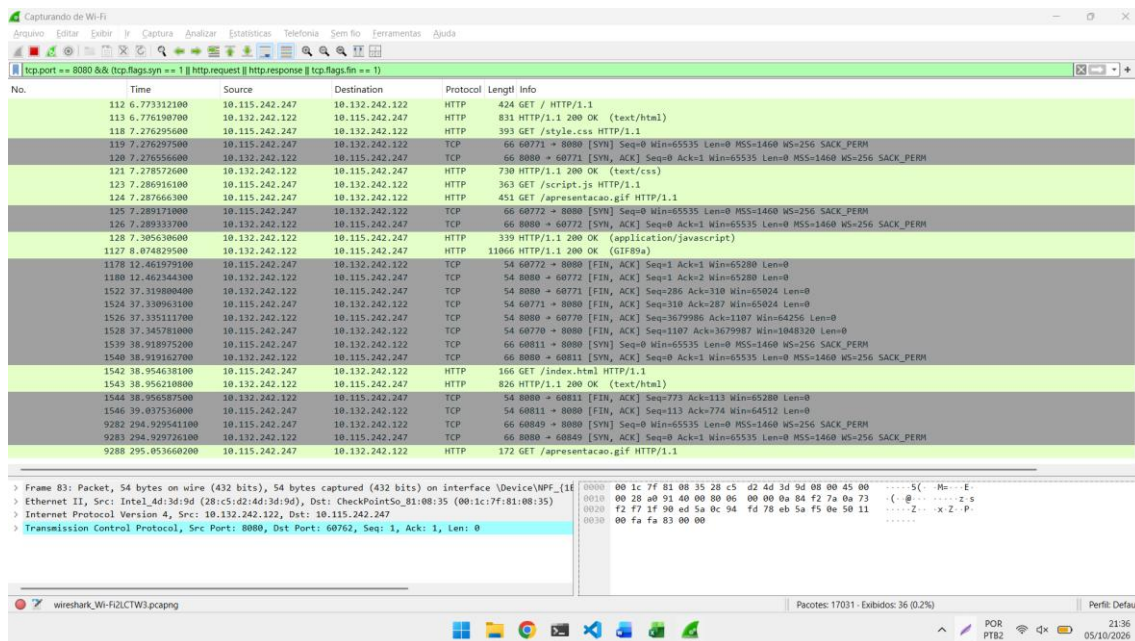
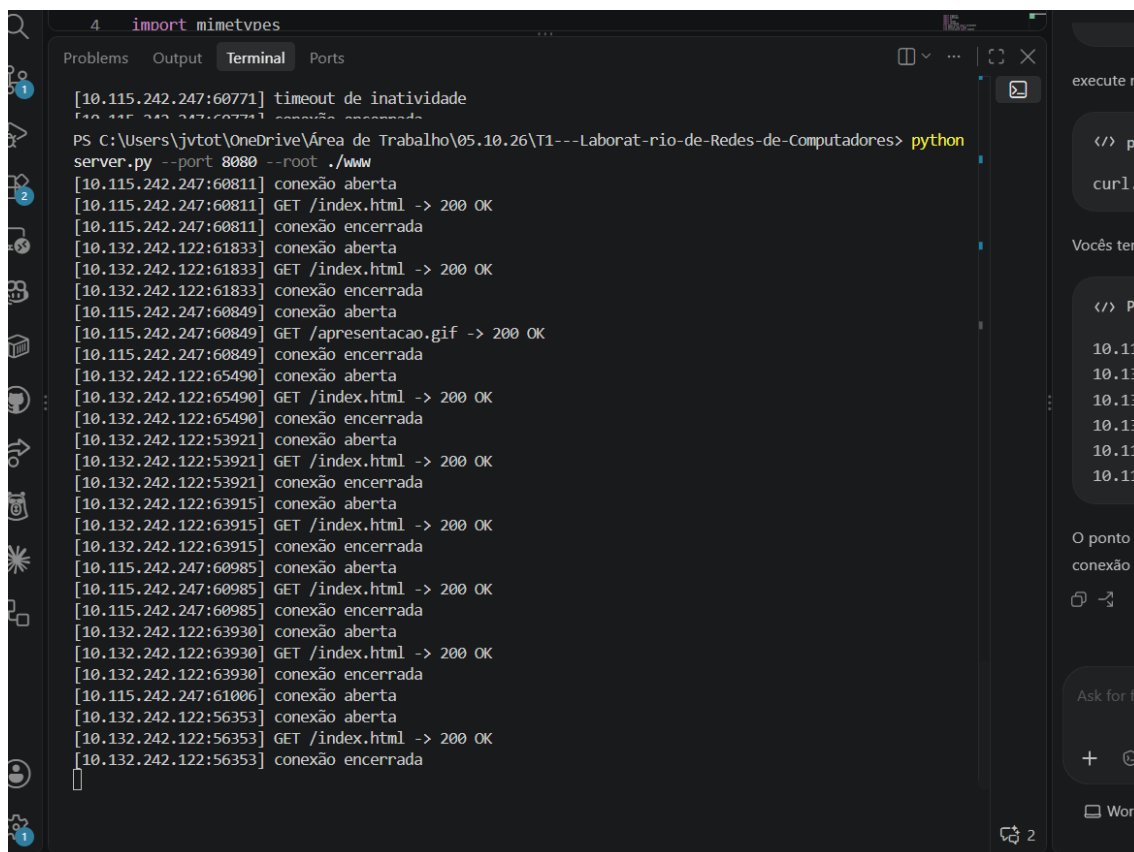


Figura 7 – Captura Wireshark durante o teste de atendimento simultâneo.

Fonte: Elaborado pelos autores (2026).

A presença de tráfego correspondente a mais de uma conexão durante o mesmo período demonstra que o servidor não permaneceu bloqueado aguardando exclusivamente a conclusão da primeira requisição. Enquanto a conexão lenta continuava ativa, a segunda conexão pôde prosseguir normalmente.

A Figura 8 apresenta a mesma situação pelo ponto de vista do servidor. No log são registrados os endereços IP e as portas de origem das conexões, permitindo observar a alternância entre conexões provenientes de 10.115.242.247 e 10.132.242.122.



```
4 import mimetypes

[10.115.242.247:60771] timeout de inatividade
[10.115.242.247:60771] conexão encerrada
PS C:\Users\jvtot\OneDrive\Área de Trabalho\05.10.26\T1---Laborat-rio-de-Redes-de-Computadores> python
server.py --port 8080 --root ./www
[10.115.242.247:60811] conexão aberta
[10.115.242.247:60811] GET /index.html -> 200 OK
[10.115.242.247:60811] conexão encerrada
[10.132.242.122:61833] conexão aberta
[10.132.242.122:61833] GET /index.html -> 200 OK
[10.132.242.122:61833] conexão encerrada
[10.115.242.247:60849] conexão aberta
[10.115.242.247:60849] GET /apresentacao.gif -> 200 OK
[10.115.242.247:60849] conexão encerrada
[10.132.242.122:65490] conexão aberta
[10.132.242.122:65490] GET /index.html -> 200 OK
[10.132.242.122:65490] conexão encerrada
[10.132.242.122:53921] conexão aberta
[10.132.242.122:53921] GET /index.html -> 200 OK
[10.132.242.122:53921] conexão encerrada
[10.132.242.122:63915] conexão aberta
[10.132.242.122:63915] GET /index.html -> 200 OK
[10.132.242.122:63915] conexão encerrada
[10.115.242.247:60985] conexão aberta
[10.115.242.247:60985] GET /index.html -> 200 OK
[10.115.242.247:60985] conexão encerrada
[10.132.242.122:63930] conexão aberta
[10.132.242.122:63930] GET /index.html -> 200 OK
[10.132.242.122:63930] conexão encerrada
[10.115.242.247:61006] conexão aberta
[10.132.242.122:56353] conexão aberta
[10.132.242.122:56353] GET /index.html -> 200 OK
[10.132.242.122:56353] conexão encerrada
```

Figura 8 – Log do servidor durante o atendimento simultâneo de conexões provenientes de máquinas distintas.

Fonte: Elaborado pelos autores (2026).

Conforme observado na Figura 8, a segunda conexão foi aceita e processada enquanto a primeira ainda permanecia aberta. A requisição ao arquivo index.html recebeu a resposta HTTP 200 OK e teve seu atendimento concluído antes da finalização da conexão lenta.

Posteriormente, a primeira requisição também pôde continuar seu processamento. Portanto, o atendimento da segunda conexão não dependeu da conclusão da primeira.

Esse comportamento é consequência direta da estratégia de concorrência adotada na implementação. Como cada conexão TCP é tratada por uma thread independente, uma thread que permanece bloqueada aguardando novos dados de um cliente não impede que outras threads processem requisições simultaneamente.

O teste demonstra, portanto, que uma requisição lenta não bloqueia as demais conexões e confirma que o servidor é capaz de atender múltiplos clientes simultaneamente.

RTT MEDIDO ENTRE AS MÁQUINAS UTILIZADAS

Para medir o Round-Trip Time (RTT) entre as máquinas utilizadas no experimento, foi analisado o tráfego TCP capturado pelo Wireshark entre o cliente 10.115.242.247 e o servidor 10.132.242.122.

Inicialmente, tentou-se realizar a medição por meio do comando ping. Entretanto, as mensagens ICMP estavam bloqueadas pela rede utilizada durante os testes, impossibilitando a obtenção do RTT dessa forma. Como alternativa, o RTT foi estimado diretamente a partir do estabelecimento das conexões TCP registradas no Wireshark.

Para cada medição, foi considerado o intervalo de tempo entre o envio do segmento SYN pela máquina cliente e o recebimento do respectivo segmento SYN, ACK enviado pelo servidor. Esse intervalo representa uma estimativa do tempo de ida e volta entre as duas máquinas, uma vez que inclui o percurso do SYN até o servidor e o retorno do SYN, ACK ao cliente.

A Figura 9 apresenta os pares de segmentos SYN e SYN, ACK utilizados para realizar as medições.

No.	Time	Source	Destination	Protocol	Length	Info
112	6.773312100	10.115.242.247	10.132.242.122	HTTP	424	GET / HTTP/1.1
113	6.776190700	10.132.242.122	10.115.242.247	HTTP	831	HTTP/1.1 200 OK (text/html)
118	7.276295600	10.115.242.247	10.132.242.122	HTTP	393	GET /style.css HTTP/1.1
119	7.276295900	10.115.242.247	10.132.242.122	TCP	66	60772 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
120	7.276556600	10.132.242.122	10.115.242.247	TCP	66	8080 → 60771 [FIN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
121	7.278572600	10.132.242.122	10.115.242.247	HTTP	730	HTTP/1.1 200 OK (text/css)
123	7.286916100	10.115.242.247	10.132.242.122	HTTP	363	GET /script.js HTTP/1.1
124	7.287666300	10.115.242.247	10.132.242.122	HTTP	451	GET /apresentacao.gif HTTP/1.1
125	7.289171000	10.115.242.247	10.132.242.122	TCP	66	60772 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
126	7.289333700	10.132.242.122	10.115.242.247	TCP	66	8080 → 60772 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
128	7.389563000	10.132.242.122	10.115.242.247	HTTP	339	HTTP/1.1 200 OK (application/javascript)
127	8.074829500	10.132.242.122	10.115.242.247	HTTP	11066	HTTP/1.1 200 OK (GIF89a)
1178	12.461979100	10.115.242.247	10.132.242.122	TCP	54	60772 → 8080 [FIN, ACK] Seq=1 Ack=1 Win=65280 Len=0
1180	12.462344300	10.132.242.122	10.115.242.247	TCP	54	8080 → 60772 [FIN, ACK] Seq=1 Ack=2 Win=65280 Len=0
1522	37.319800400	10.132.242.122	10.115.242.247	TCP	54	8080 → 60771 [FIN, ACK] Seq=286 Ack=310 Win=65024 Len=0
1524	37.330063100	10.115.242.247	10.132.242.122	TCP	54	60772 → 8080 [FIN, ACK] Seq=310 Ack=287 Win=65024 Len=0
1526	37.335111700	10.132.242.122	10.115.242.247	TCP	54	8080 → 60770 [FIN, ACK] Seq=3679986 Ack=1107 Win=64256 Len=0
1528	37.345781000	10.115.242.247	10.132.242.122	TCP	54	60770 → 8080 [FIN, ACK] Seq=1107 Ack=3679987 Win=1048320 Len=0
1539	38.918975200	10.115.242.247	10.132.242.122	TCP	66	60811 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
1540	38.919162700	10.132.242.122	10.115.242.247	TCP	66	8080 → 60811 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
1542	38.954638100	10.115.242.247	10.132.242.122	HTTP	166	GET /index.html HTTP/1.1
1543	38.956210800	10.132.242.122	10.115.242.247	HTTP	826	HTTP/1.1 200 OK (text/html)
1544	38.956875000	10.132.242.122	10.115.242.247	TCP	54	8080 → 60811 [FIN, ACK] Seq=773 Ack=113 Win=65280 Len=0
1546	39.037536800	10.115.242.247	10.132.242.122	TCP	54	60811 → 8080 [FIN, ACK] Seq=113 Ack=774 Win=64512 Len=0
9282	294.929541100	10.115.242.247	10.132.242.122	TCP	66	60849 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
9283	294.929726100	10.132.242.122	10.115.242.247	TCP	66	8080 → 60849 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
9288	295.055660200	10.115.242.247	10.132.242.122	HTTP	172	GET /apresentacao.gif HTTP/1.1

> Frame 83: Packet, 54 bytes on wire (432 bits), 54 bytes captured (432 bits) on interface \Device\NPF...
> Ethernet II, Src: Intel_4d:3d:9d (28:c5:d2:4d:3d:9d), Dst: CheckPointSo_81:08:35 (00:1c:7f:81:08:35)
> Internet Protocol Version 4, Src: 10.132.242.122, Dst: 10.115.242.247
> Transmission Control Protocol, Src Port: 8080, Dst Port: 60762, Seq: 1, Ack: 1, Len: 0

0000 00 1c 7f 81 08 35 28 c5 d2 4d 3d 9d 08 00 45 005(-)M:-E
0010 00 28 a0 91 40 00 00 00 00 0a 84 f2 7a 0a 732.....z:s
0020 f2 7f 1f 90 ed 5a 0c 94 fd 78 ab 5a f5 0e 50 112->x.2-P
0030 00 fa 83 00 00

Figura 9 – Segmentos SYN e SYN, ACK utilizados para estimar o RTT entre o cliente 10.115.242.247 e o servidor 10.132.242.122.

Fonte: Elaborado pelos autores (2026).

Foram considerados quatro estabelecimentos de conexão presentes na captura.

Na primeira medição, o segmento SYN foi registrado no instante 7,276297500 s e o respectivo SYN, ACK no instante 7,276556600 s. Assim:

$$RTT1 = 7,276556600 - 7,276297500$$

$$RTT1 = 0,000259100 \text{ s}$$

$$RTT1 = 0,2591 \text{ ms}$$

Na segunda medição, o SYN foi registrado em 7,289171000 s e o SYN, ACK em 7,289333700 s:

$$RTT2 = 7,289333700 - 7,289171000$$

$$RTT2 = 0,000162700 \text{ s}$$

$$RTT2 = 0,1627 \text{ ms}$$

Na terceira medição, o SYN foi registrado em 38,918975200 s e o SYN, ACK em 38,919162700 s:

$$RTT3 = 38,919162700 - 38,918975200$$

$$RTT3 = 0,000187500 \text{ s}$$

$$RTT3 = 0,1875 \text{ ms}$$

Na quarta medição, o SYN foi registrado em 294,929541100 s e o SYN, ACK em 294,929726100 s:

$$RTT4 = 294,929726100 - 294,929541100$$

$$RTT4 = 0,000185000 \text{ s}$$

$$RTT4 = 0,1850 \text{ ms}$$

A média das quatro medições foi:

$$RTT \text{ médio} = (0,2591 + 0,1627 + 0,1875 + 0,1850) / 4$$

$$RTT \text{ médio} = 0,198575 \text{ ms}$$

Portanto, o RTT médio estimado entre as máquinas utilizadas foi de aproximadamente 0,20 ms. O menor valor observado foi de 0,1627 ms e o maior foi de 0,2591 ms.

Por se tratar de uma medição baseada no intervalo SYN/SYN, ACK, o valor também pode incluir uma pequena parcela do tempo de processamento da pilha TCP do servidor. Ainda assim, esse método fornece uma estimativa adequada do RTT entre as duas máquinas no ambiente utilizado durante os testes.

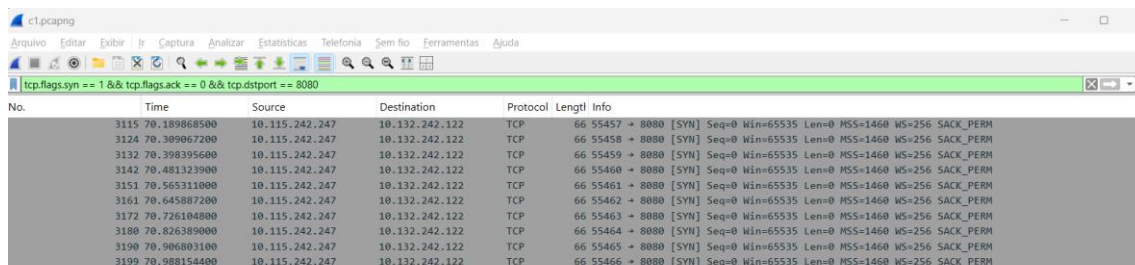
TABELA COMPARATIVA C1 × C2

Para avaliar o impacto das conexões persistentes no tráfego de rede, foram realizadas dez requisições HTTP sequenciais ao arquivo teste.txt em dois cenários distintos.

No cenário C1, cada uma das dez requisições utilizou uma nova conexão TCP. Dessa forma, cada requisição exigiu o estabelecimento de uma nova conexão, sua utilização para a transferência HTTP e o posterior encerramento.

No cenário C2, as mesmas dez requisições foram realizadas utilizando uma única conexão TCP persistente. A conexão foi estabelecida uma única vez e reutilizada para todas as requisições antes de ser encerrada.

A Figura 10 apresenta os segmentos SYN observados no cenário C1. O filtro utilizado seleciona apenas os segmentos responsáveis pelo início de novas conexões TCP, permitindo identificar dez conexões diferentes.



The image shows a Wireshark packet capture window titled 'c1.pcapng'. The filter bar at the top contains the expression 'tcp.flags.syn == 1 && tcp.flags.ack == 0 && tcp.dport == 8080'. The packet list below shows 10 packets, all of which are TCP SYN segments. Each packet has a source IP of 10.115.242.247 and a destination IP of 10.132.242.122. The sequence numbers for the SYN segments are 6655457, 6655458, 6655459, 6655460, 6655461, 6655462, 6655463, 6655464, 6655465, and 6655466. The window size is 256 bytes for all segments.

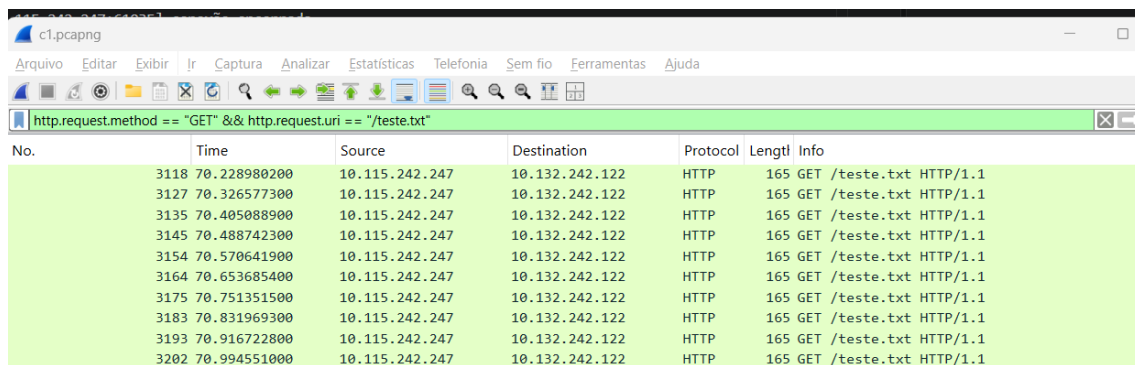
No.	Time	Source	Destination	Protocol	Length	Info
3115	70.189868500	10.115.242.247	10.132.242.122	TCP	66	55457 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
3124	70.309067200	10.115.242.247	10.132.242.122	TCP	66	55458 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
3132	70.398395600	10.115.242.247	10.132.242.122	TCP	66	55459 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
3142	70.481323900	10.115.242.247	10.132.242.122	TCP	66	55460 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
3151	70.565311000	10.115.242.247	10.132.242.122	TCP	66	55461 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
3161	70.645887200	10.115.242.247	10.132.242.122	TCP	66	55462 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
3172	70.736104800	10.115.242.247	10.132.242.122	TCP	66	55463 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
3180	70.826389000	10.115.242.247	10.132.242.122	TCP	66	55464 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
3190	70.906803100	10.115.242.247	10.132.242.122	TCP	66	55465 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
3199	70.988154400	10.115.242.247	10.132.242.122	TCP	66	55466 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM

Figura 10 – Segmentos SYN correspondentes às dez conexões TCP estabelecidas no cenário C1.

Fonte: Elaborado pelos autores (2026).

Como cada requisição do cenário C1 utilizou uma nova conexão, foram observados dez segmentos SYN de abertura e, conseqüentemente, dez handshakes TCP completos.

A Figura 11 apresenta as estatísticas da captura correspondente ao cenário C1.



The image shows a Wireshark packet capture window titled 'c1.pcapng'. The filter bar at the top contains the expression 'http.request.method == "GET" && http.request.uri == "/teste.txt"'. The packet list below shows 10 packets, all of which are HTTP GET requests. Each packet has a source IP of 10.115.242.247 and a destination IP of 10.132.242.122. The sequence numbers for the GET requests are 6655457, 6655458, 6655459, 6655460, 6655461, 6655462, 6655463, 6655464, 6655465, and 6655466. The window size is 256 bytes for all segments.

No.	Time	Source	Destination	Protocol	Length	Info
3118	70.228980200	10.115.242.247	10.132.242.122	HTTP	165	GET /teste.txt HTTP/1.1
3127	70.326577300	10.115.242.247	10.132.242.122	HTTP	165	GET /teste.txt HTTP/1.1
3135	70.405088900	10.115.242.247	10.132.242.122	HTTP	165	GET /teste.txt HTTP/1.1
3145	70.488742300	10.115.242.247	10.132.242.122	HTTP	165	GET /teste.txt HTTP/1.1
3154	70.570641900	10.115.242.247	10.132.242.122	HTTP	165	GET /teste.txt HTTP/1.1
3164	70.653685400	10.115.242.247	10.132.242.122	HTTP	165	GET /teste.txt HTTP/1.1
3175	70.751351500	10.115.242.247	10.132.242.122	HTTP	165	GET /teste.txt HTTP/1.1
3183	70.831969300	10.115.242.247	10.132.242.122	HTTP	165	GET /teste.txt HTTP/1.1
3193	70.916722800	10.115.242.247	10.132.242.122	HTTP	165	GET /teste.txt HTTP/1.1
3202	70.994551000	10.115.242.247	10.132.242.122	HTTP	165	GET /teste.txt HTTP/1.1

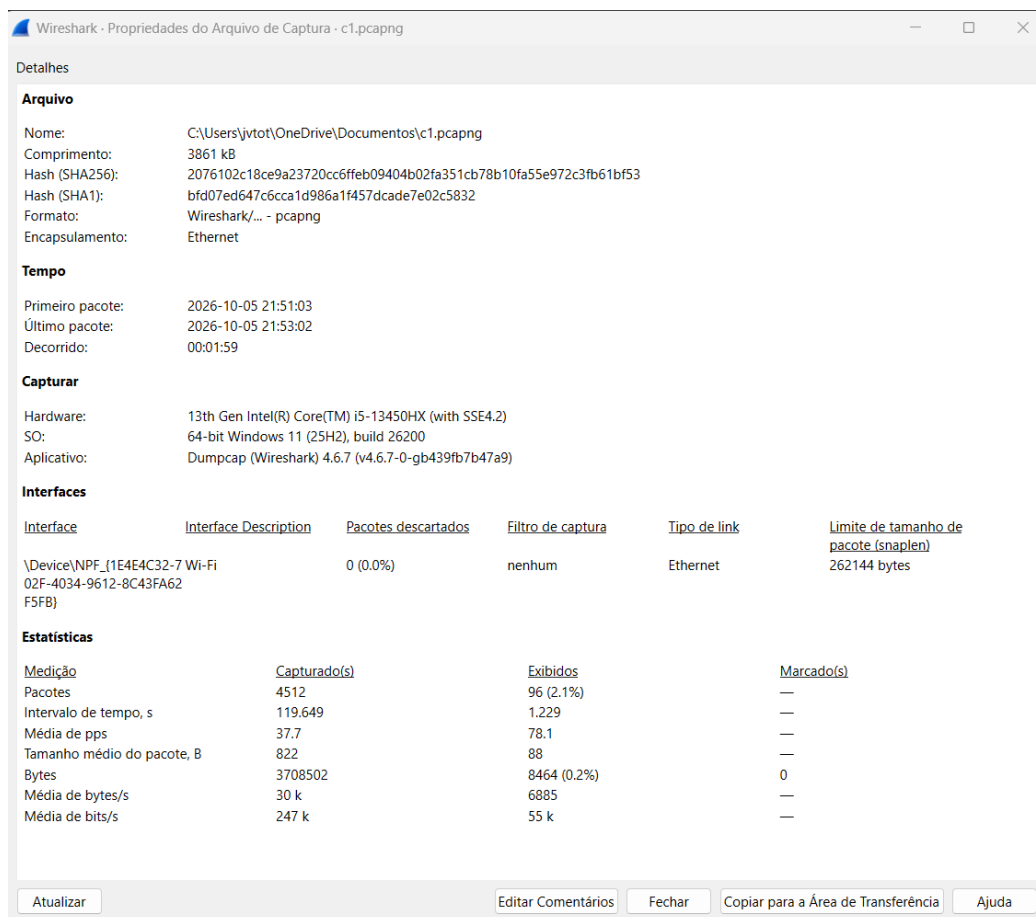


Figura 11 – Estatísticas da captura do cenário C1, com uma nova conexão TCP para cada requisição.

Fonte: Elaborado pelos autores (2026).

Considerando apenas os pacotes exibidos pelo filtro correspondente ao experimento, o cenário C1 apresentou 96 pacotes, totalizando 8.464 bytes e um intervalo de 1,229 segundo.

No cenário C2, as dez requisições utilizaram uma única conexão persistente. A Figura 12 apresenta a captura correspondente. É possível observar inicialmente apenas um handshake TCP, formado pelos segmentos SYN, SYN-ACK e ACK. Em seguida, as dez requisições GET e suas respectivas respostas HTTP/1.1 200 OK são transportadas pela mesma conexão antes do encerramento.

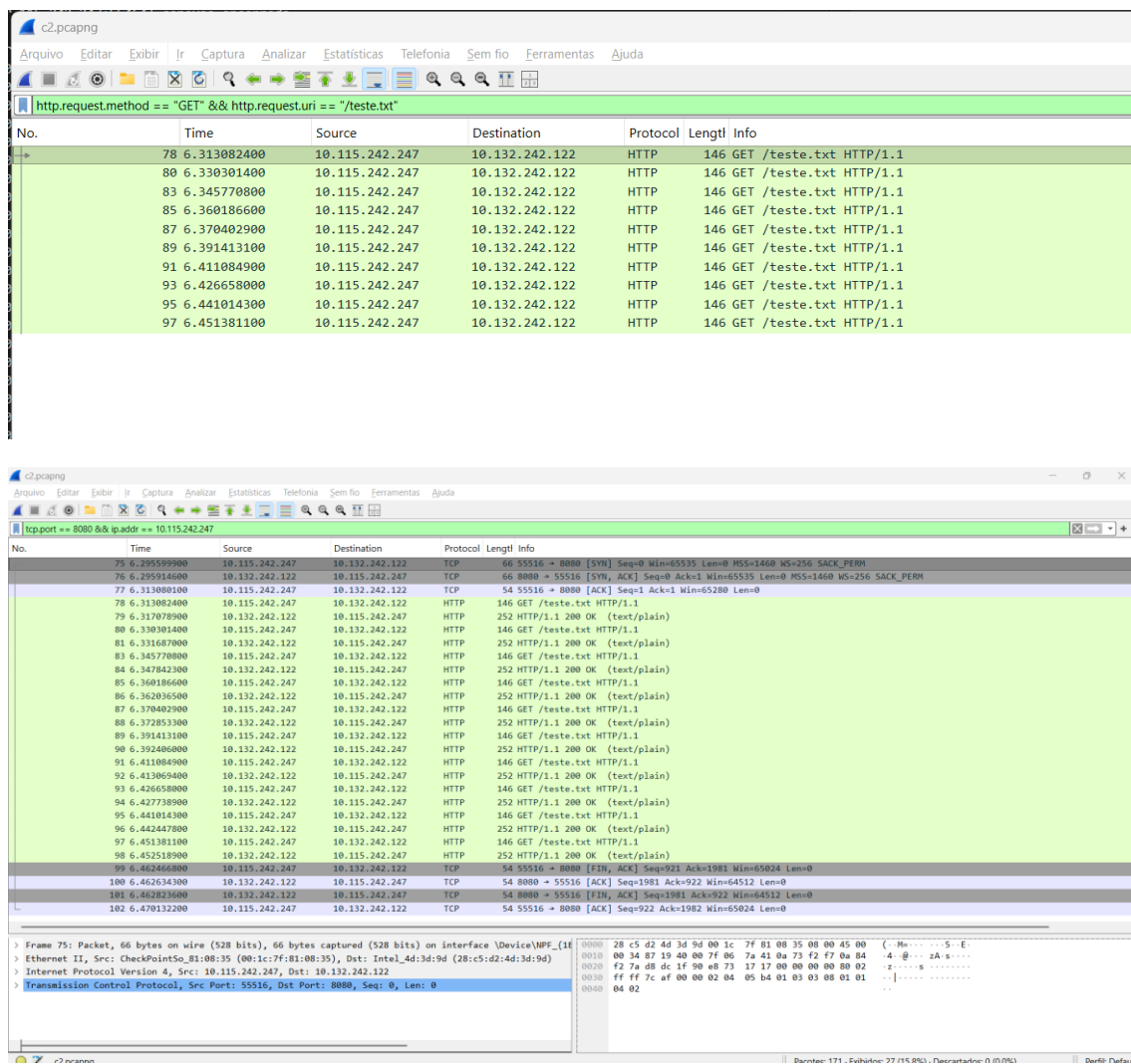


Figura 12 – Dez requisições HTTP realizadas sobre uma única conexão TCP persistente no cenário C2.

Fonte: Elaborado pelos autores (2026).

A Figura 13 apresenta as estatísticas da captura do cenário C2.

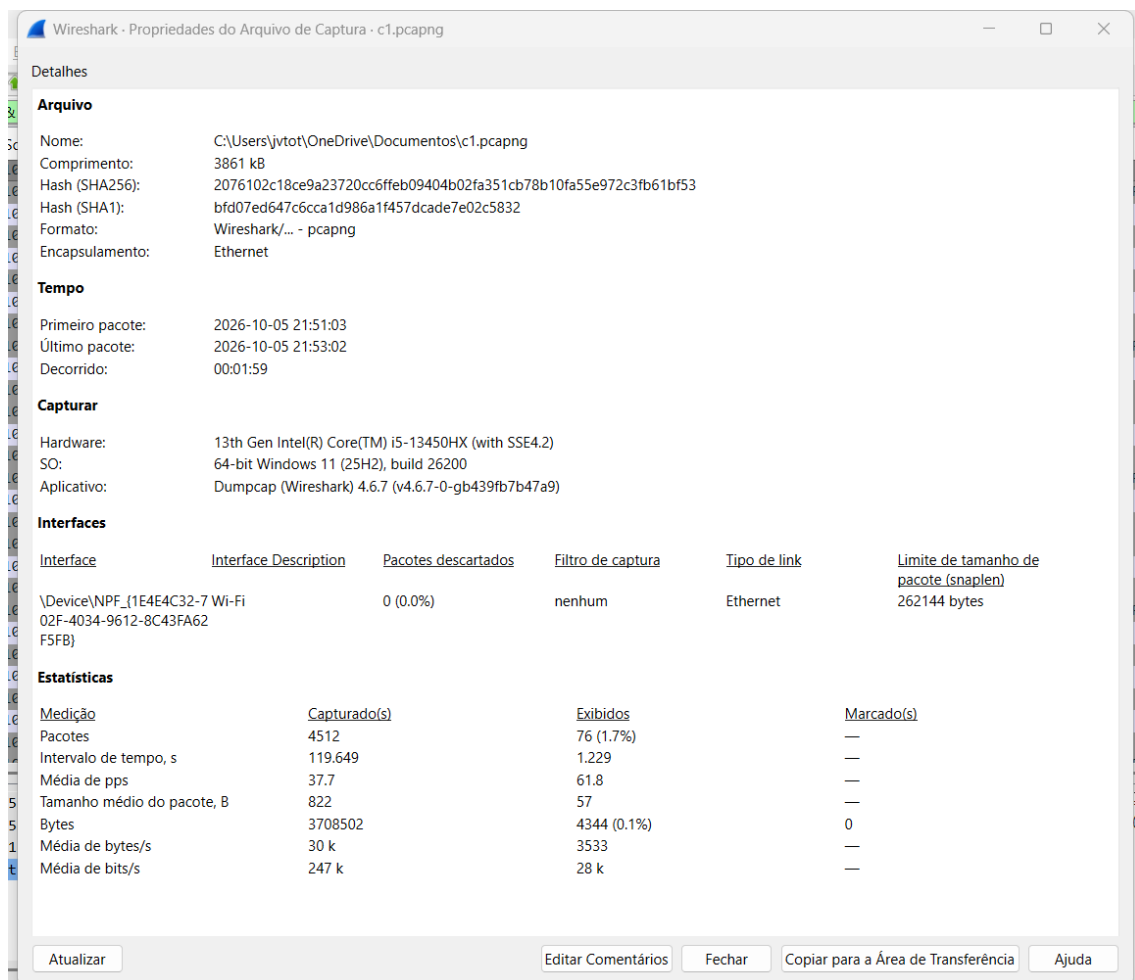


Figura 13 – Estatísticas da captura do cenário C2 utilizando conexão TCP persistente.

Fonte: Elaborado pelos autores (2026).

No cenário C2 foram observados 27 pacotes, totalizando 4.382 bytes e um intervalo de 0,175 segundo. Como todas as dez requisições utilizaram a mesma conexão, foi necessário apenas um handshake TCP.

A Tabela 2 resume os resultados obtidos nos dois cenários.

Tabela 2 – Comparação entre novas conexões e conexão persistente

Métrica	C1 – novas conexões	C2 – conexão persistente	Economia com C2
Handshakes TCP completos	10	1	-
Total de pacotes	96	27	71,88%
Total de bytes	8.464 bytes	4.382 bytes	48,23%
Tempo total	1,229 s	0,175 s	—

Fonte: Elaborado pelos autores (2026).

A economia percentual de pacotes proporcionada pela conexão persistente foi calculada por:

$$\text{Economia de pacotes} = ((96 - 27) / 96) \times 100$$

$$\text{Economia de pacotes} = 71,88\%$$

Para o volume de dados trafegados:

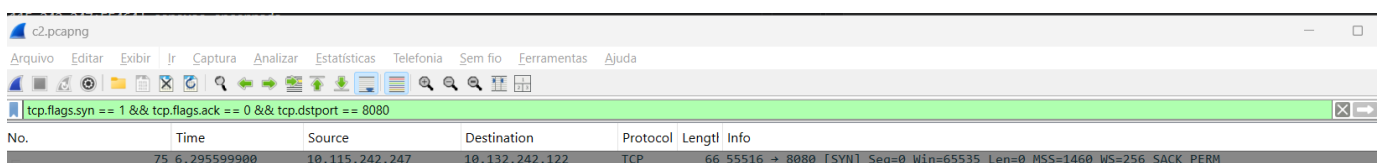
$$\text{Economia de bytes} = ((8.464 - 4.382) / 8.464) \times 100$$

$$\text{Economia de bytes} = 48,23\%$$

Portanto, para a mesma quantidade de dez requisições HTTP, o uso de uma conexão persistente reduziu o número total de pacotes de 96 para 27, correspondendo a uma economia de 71,88%. O volume de dados trafegados também foi reduzido de 8.464 para 4.382 bytes, representando uma economia de 48,23%.

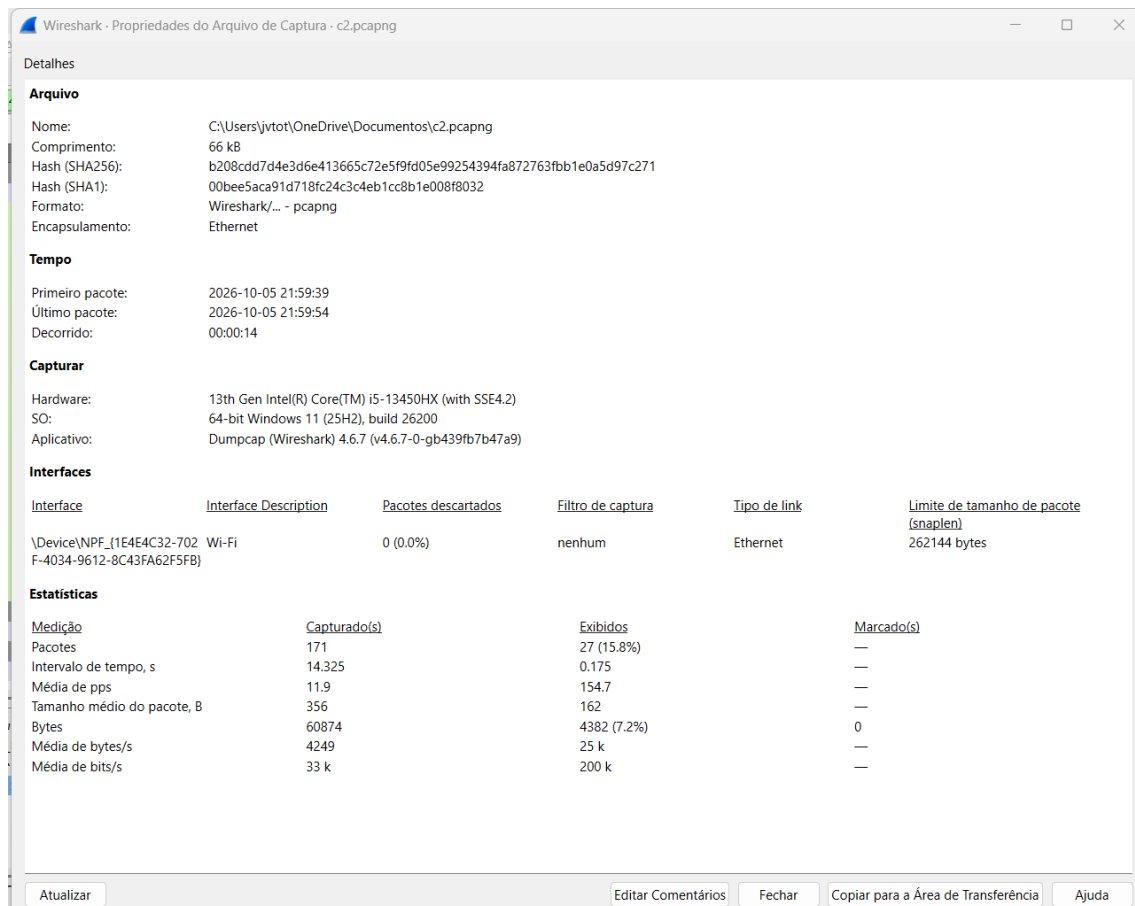
Além disso, o número de handshakes TCP foi reduzido de dez para apenas um, uma vez que a mesma conexão foi reutilizada para todas as requisições. O tempo total observado também diminuiu de 1,229 segundo para 0,175 segundo.

Os resultados demonstram que a conexão persistente reduz o tráfego necessário para realizar requisições sucessivas, principalmente por evitar a repetição dos procedimentos de estabelecimento e encerramento de uma nova conexão TCP para cada requisição.



The screenshot shows the Wireshark network protocol analyzer interface. The title bar indicates the file is 'c2.pcapng'. The menu bar includes 'Arquivo', 'Editar', 'Exibir', 'Ir', 'Captura', 'Analisar', 'Estatísticas', 'Telefonia', 'Sem fio', 'Ferramentas', and 'Ajuda'. The toolbar contains various icons for file operations, capture, and analysis. The packet list pane shows a single packet with the filter 'tcp.flags.syn == 1 && tcp.flags.ack == 0 && tcp.dstport == 8080'. The packet details pane shows the selected packet (No. 75) at time 6.295599900, from source 10.115.242.247 to destination 10.132.242.122, protocol TCP, length 66 bytes. The packet info pane shows the details: 66 55516 -> 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM.

No.	Time	Source	Destination	Protocol	Length	Info
75	6.295599900	10.115.242.247	10.132.242.122	TCP	66	55516 -> 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM



ANÁLISE DO OVERHEAD DE CONEXÃO

Para quantificar o overhead causado pela abertura e pelo encerramento das conexões TCP no cenário C1, foi analisado o tráfego correspondente às dez requisições realizadas com uma nova conexão para cada acesso ao arquivo teste.txt.

Como cada uma das dez requisições utilizou uma conexão TCP diferente, foram realizados dez processos completos de estabelecimento e dez processos de encerramento de conexão.

O estabelecimento de uma conexão TCP utiliza o handshake de três vias, formado pelos segmentos SYN, SYN-ACK e ACK. Dessa forma, para as dez conexões do cenário C1 foram necessários:

$$10 \text{ conexões} \times 3 \text{ pacotes} = 30 \text{ pacotes}$$

Na captura, esses 30 pacotes utilizados nos handshakes corresponderam a aproximadamente 1.860 bytes.

Após a transferência de cada resposta HTTP, cada conexão também precisou ser encerrada. Em condições normais, o encerramento TCP utiliza quatro

segmentos, correspondentes à troca de FIN/ACK e suas respectivas confirmações. Para as dez conexões foram necessários:

10 conexões × 4 pacotes = 40 pacotes

Esses pacotes de encerramento corresponderam a aproximadamente 2.160 bytes.

Considerando apenas o custo diretamente relacionado à abertura e ao fechamento das dez conexões, obtém-se:

Abertura das conexões:

30 pacotes

aproximadamente 1.860 bytes

Encerramento das conexões:

40 pacotes

aproximadamente 2.160 bytes

Total de abertura e encerramento:

70 pacotes

aproximadamente 4.020 bytes

Para complementar a análise, foi aplicado à captura do cenário C1 o seguinte filtro no Wireshark:

```
tcp.port == 8080 && ip.addr == 10.115.242.247 && tcp.len == 0
```

Esse filtro seleciona segmentos TCP sem dados de aplicação, incluindo os pacotes envolvidos no estabelecimento, nas confirmações e no encerramento das conexões.

A Figura 14 apresenta as estatísticas obtidas com esse filtro.

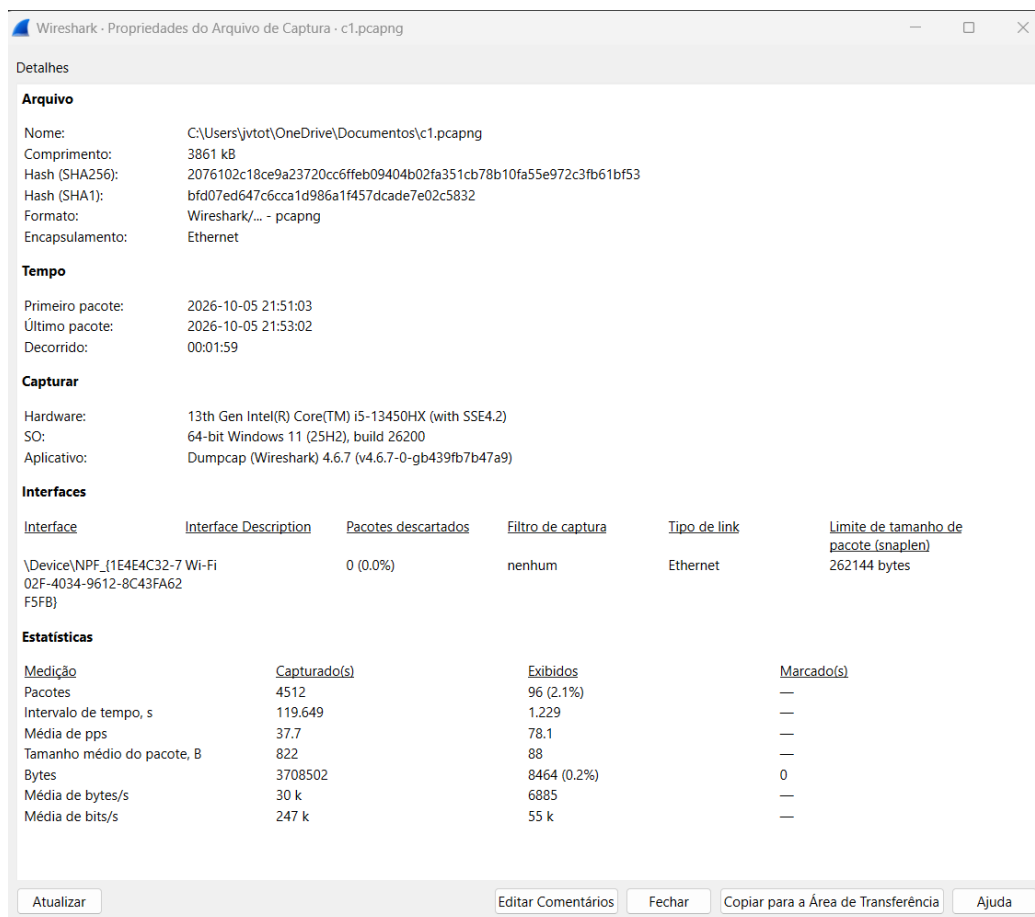


Figura 14 – Tráfego de controle TCP observado no cenário C1.

Fonte: Elaborado pelos autores (2026).

Na captura filtrada foram observados 76 pacotes de controle, totalizando 4.344 bytes. Esse valor é ligeiramente superior aos 70 pacotes e 4.020 bytes correspondentes estritamente à abertura e ao encerramento das dez conexões porque a captura também contém segmentos adicionais de controle, como confirmações, retransmissões e atualizações relacionadas ao funcionamento do TCP.

Portanto, considerando especificamente os procedimentos de abertura e encerramento das conexões TCP no cenário C1, foram gastos aproximadamente 70 pacotes e 4.020 bytes. Considerando todo o tráfego de controle TCP efetivamente observado na captura, foram registrados 76 pacotes e 4.344 bytes.

A diferença evidencia o overhead provocado pela utilização de uma nova conexão para cada requisição. No cenário com conexão persistente, grande parte desse custo é evitada, pois o mesmo estabelecimento TCP pode ser reutilizado para várias requisições HTTP.

ANÁLISE EM FUNÇÃO DO RTT

A diferença entre os cenários C1 e C2 também pode ser analisada em função do Round-Trip Time (RTT) medido entre as máquinas.

No cenário C1, cada uma das dez requisições HTTP utilizou uma nova conexão TCP. Dessa forma, foram necessários dez handshakes TCP completos. No cenário C2, as dez requisições reutilizaram uma única conexão persistente, exigindo apenas um handshake.

O estabelecimento de uma conexão TCP utiliza o mecanismo de três vias, composto por SYN, SYN-ACK e ACK. Antes que o cliente possa prosseguir com a requisição HTTP em uma nova conexão, é necessário aguardar aproximadamente um RTT para que o segmento SYN chegue ao servidor e o SYN-ACK retorne ao cliente.

Portanto:

C1 = 10 handshakes $\approx$ 10 RTTs de estabelecimento

C2 = 1 handshake $\approx$ 1 RTT de estabelecimento

Diferença = 10 RTTs - 1 RTT

Diferença = 9 RTTs

Assim, o cenário C1 apresenta um custo adicional de aproximadamente 9 RTTs apenas devido aos nove estabelecimentos de conexão que poderiam ter sido evitados com a reutilização da conexão TCP.

O RTT médio estimado anteriormente entre o cliente 10.115.242.247 e o servidor 10.132.242.122 foi de:

RTT médio = 0,198575 ms

Logo, o custo teórico correspondente aos nove handshakes adicionais é:

$9 \times 0,198575 \text{ ms} = 1,787175 \text{ ms}$

Portanto, aproximadamente 1,79 ms do tempo adicional do cenário C1 podem ser diretamente associados ao custo de propagação e estabelecimento das nove conexões TCP adicionais, considerando o RTT medido no experimento.

O tempo total observado nos dois cenários foi:

$C1 = 1,229 \text{ s}$

$C2 = 0,175 \text{ s}$

A diferença de tempo observada foi:

$1,229 \text{ s} - 0,175 \text{ s} = 1,054 \text{ s}$

ou:

$1,054 \text{ s} = 1.054 \text{ ms}$

Comparando esse valor com o custo estimado dos handshakes:

Custo dos 9 RTTs $\approx 1,79 \text{ ms}$

Diferença total observada $\approx 1.054 \text{ ms}$

Isso demonstra que, no ambiente utilizado para os testes, o RTT da rede era muito baixo e, portanto, o custo de propagação dos handshakes representa apenas uma pequena parcela da diferença total de tempo entre C1 e C2.

A maior parte da diferença observada está associada a outros custos introduzidos pelo uso repetido de novas conexões, como criação e encerramento de sockets, processamento no cliente e no servidor, escalonamento do sistema operacional e demais operações necessárias para executar cada requisição separadamente.

Ainda assim, a relação com o RTT é importante: cada nova conexão adiciona aproximadamente um RTT antes que a comunicação HTTP possa prosseguir. Assim, em uma rede com maior latência, o impacto dos nove handshakes adicionais seria proporcionalmente maior.

Dessa forma, o cenário C1 gasta aproximadamente 9 RTTs a mais que o cenário C2 exclusivamente devido à necessidade de estabelecer nove conexões TCP adicionais. O cenário C2 evita esse custo ao reutilizar uma única conexão persistente para todas as dez requisições.

CONCLUSÃO

Os resultados obtidos demonstram que a utilização de conexões HTTP persistentes reduz tanto o tráfego de controle TCP quanto o tempo necessário para realizar sucessivas requisições ao mesmo servidor.

No cenário C1, cada uma das dez requisições exigiu o estabelecimento de uma nova conexão TCP, enquanto no cenário C2 todas as requisições reutilizaram uma única conexão. Dessa forma, o cenário C1 realizou nove handshakes TCP adicionais em relação ao cenário C2.

Como cada novo estabelecimento de conexão adiciona aproximadamente um RTT antes que a requisição HTTP possa prosseguir, a diferença entre os cenários cresce à medida que aumenta a latência da rede. Para N requisições realizadas em conexões separadas, em comparação com uma única conexão persistente, o custo adicional de estabelecimento pode ser aproximado por:

$$\text{Custo adicional} \approx (N - 1) \times \text{RTT}$$

No experimento realizado:

$$N = 10$$

$$\text{Custo adicional} \approx (10 - 1) \times \text{RTT}$$

$$\text{Custo adicional} \approx 9 \times \text{RTT}$$

Como o RTT medido entre as máquinas foi muito baixo, aproximadamente 0,20 ms, o impacto exclusivamente relacionado ao tempo de propagação dos handshakes também foi pequeno no ambiente de teste.

Entretanto, em uma rede com RTT mais elevado, a vantagem da conexão persistente seria significativamente maior. Isso ocorre, por exemplo, quando cliente e servidor estão geograficamente distantes, quando a comunicação atravessa diversos enlaces de rede ou quando a infraestrutura apresenta maior latência.

Como exemplo, se o RTT fosse de 50 ms, os nove handshakes adicionais do cenário C1 representariam aproximadamente:

$$9 \times 50 \text{ ms} = 450 \text{ ms}$$

Se o RTT fosse de 100 ms:

$$9 \times 100 \text{ ms} = 900 \text{ ms}$$

Portanto, quanto maior o RTT e maior o número de requisições que exigiram novas conexões TCP, maior será o benefício obtido com a reutilização de uma conexão persistente.

Além da redução do tempo associado aos handshakes, os resultados do experimento também mostraram redução no número de pacotes e no volume de bytes trafegados. Assim, a conexão persistente apresenta vantagens tanto em eficiência de rede quanto em tempo de comunicação, sendo especialmente vantajosa em ambientes com alta latência e várias requisições sucessivas entre o mesmo cliente e servidor.